

Image Classification with Convolutional Neural Networks: A Study on the CIFAR- 10 Dataset

Machine Learning

Contents

1. Introduction	4
1.1 Background.....	4
1.2 Project Objective.....	4
1.3 Scope.....	4
2. Dataset Overview and Data Preparation.....	4
2.1 Dataset Description	4
2.2 Data Preprocessing.....	5
2.3 Data Augmentation.....	5
3. Validation Strategy and Rationale	5
3.1 Role of the Validation Set.....	5
3.2 Early Stopping and Monitoring.....	5
4. Model Architecture and Hyperparameter Design.....	6
4.1 CNN Structure.....	6
4.2 Regularisation and Optimisation.....	7
4.3 Confusion Metrix	8
5. Training Strategy	8
5.1 Epochs and Learning Policy	8
5.2 Callbacks and Regularisation	8
5.3 Hardware and Environment.....	9
6. Performance Evaluation and Results.....	9
6.1 Comparative Accuracy:	9
6.3 Inference Example	9
7. Comparative Discussion.....	10
7.1 Comparison with Classical ML (Track 1).....	10
7.2 Comparison with Advanced ML (Track 3)	10
7.3 Strengths and Limitations.....	10
8. Conclusions and Lessons Learned	11
8.1 Key Findings	11
8.2 Lessons Learned	11
8.3 Future Work	11
9. References.....	12

Table of Figures:

Figure 1: Sample images from each CIFAR-10 class	4
Figure 2: Training and validation accuracy/loss curves	6
Figure 3: CNN architecture diagram showing layers and parameters	7
Figure 4: Confusion matrix	8
Figure 5: Learning rate changes over epochs to show schedule impact	9
Figure 6: Comparative Accuracy	9
Figure 7: Prediction	10

1. Introduction

1.1 Background

Artificial Intelligence (AI) is transforming the way computers perceive and interpret the world. One of its most impactful applications is object recognition, where algorithms identify and categorise images into predefined classes. Convolutional Neural Networks (CNNs) have proven highly effective for this task, outperforming traditional methods that rely on manual feature extraction.

1.2 Project Objective

The purpose of this project is to design, train, and evaluate a CNN for object recognition on the CIFAR-10 dataset, which consists of ten image classes such as airplane, automobile, bird, and ship. The goal is to achieve high classification accuracy while maintaining model interpretability and robustness.

1.3 Scope

This report focuses on the Deep Learning Track (Track 2) of the project, using convolutional neural networks and data augmentation. The evaluation emphasises training performance, generalisation ability, and comparison with other machine learning approaches.

2. Dataset Overview and Data Preparation

2.1 Dataset Description

The CIFAR-10 dataset is a standard benchmark in computer vision research. It contains 60,000 coloured images ($32 \times 32 \times 3$) divided into ten classes: *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship*, and *truck*.

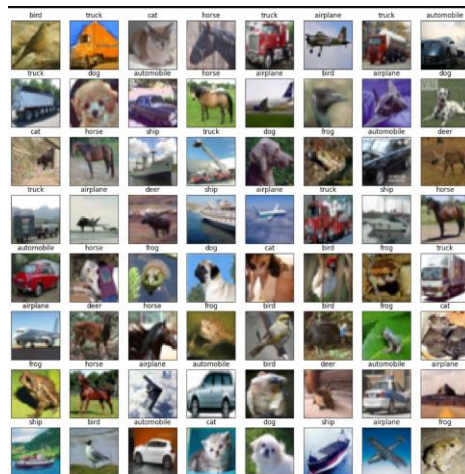


Figure 1: Sample images from each CIFAR-10 class

There are 50,000 training images and 10,000 test images, each class represented equally with 6,000 samples.

2.2 Data Preprocessing

To prepare the dataset for training:

- All pixel values were converted to float32 and normalised using the training set's mean and standard deviation.
- The training set was split into 45,000 for training and 5,000 for validation to monitor generalisation.
- The validation set was stratified to preserve class distribution.

2.3 Data Augmentation

To increase data diversity and reduce overfitting, several augmentation techniques were applied:

- Random rotations ($\pm 15^\circ$)
- Width and height shifts ($\pm 12\%$)
- Horizontal flips
- Zoom ($\pm 10\%$) and brightness changes
- Channel shift (0.1)

These transformations were implemented using Keras' ImageDataGenerator, generating unique images in each epoch.

3. Validation Strategy and Rationale

3.1 Role of the Validation Set

A dedicated validation set was used to tune hyperparameters and evaluate model performance during training. This prevents the model from overfitting to training data and ensures that it learns general patterns rather than memorising examples.

3.2 Early Stopping and Monitoring

Early stopping was applied to terminate training when validation loss stopped improving for 40 epochs. The ReduceLROnPlateau callback lowered the learning rate by half whenever performance stagnated for ten epochs, helping the model escape local minima.

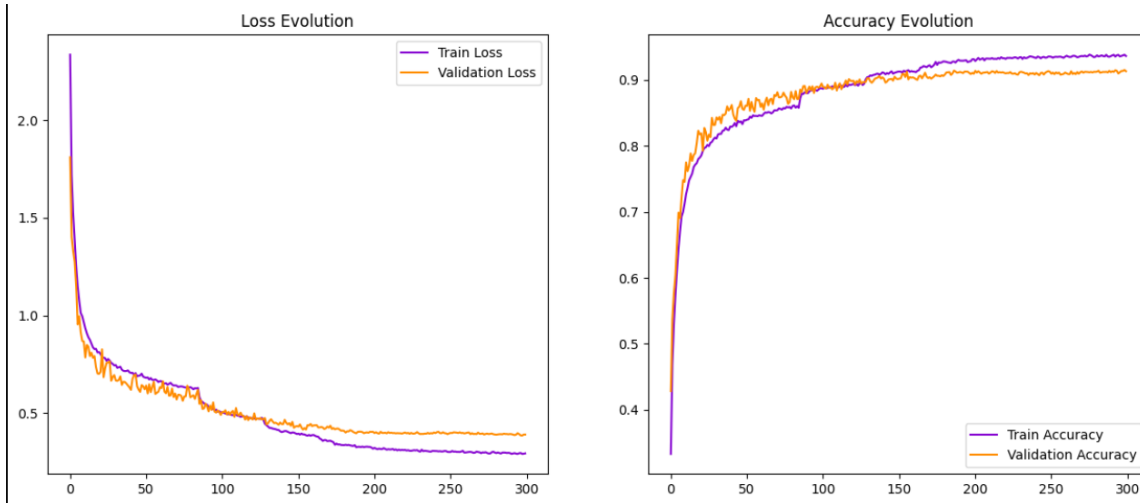


Figure 2: Training and validation accuracy/loss curves

4. Model Architecture and Hyperparameter Design

4.1 CNN Structure

The custom CNN consisted of **four convolutional blocks** followed by a fully connected classification layer.

Architecture Overview:

1. **Block 1:** Conv(32) → BatchNorm → Conv(32) → BatchNorm → MaxPool(2) → Dropout(0.2)
2. **Block 2:** Conv(64) → BatchNorm → Conv(64) → BatchNorm → MaxPool(2) → Dropout(0.3)
3. **Block 3:** Conv(128) → BatchNorm → Conv(128) → BatchNorm → MaxPool(2) → Dropout(0.4)
4. **Block 4:** Conv(256) → BatchNorm → Conv(256) → BatchNorm → MaxPool(2) → Dropout(0.5)
5. **Output:** Flatten → Dense(10, Softmax)

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 16, 16, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 8, 8, 128)	512
conv2d_5 (Conv2D)	(None, 8, 8, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 8, 8, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
conv2d_6 (Conv2D)	(None, 4, 4, 256)	295,168
batch_normalization_6 (BatchNormalization)	(None, 4, 4, 256)	1,024
conv2d_7 (Conv2D)	(None, 4, 4, 256)	590,080
batch_normalization_7 (BatchNormalization)	(None, 4, 4, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 2, 2, 256)	0
dropout_3 (Dropout)	(None, 2, 2, 256)	0
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 10)	10,250

Figure 3: CNN architecture diagram showing layers and parameters

4.2 Regularisation and Optimisation

- **L2 regularisation** ($\lambda = 1e-4$) was applied to convolution layers to control weight magnitude.
- **Batch Normalisation** after each convolution improved stability.
- **Dropout layers** reduced overfitting progressively through deeper layers.
- **Optimiser:** Adam (learning rate = 5×10^{-4})
- **Loss Function:** Categorical Cross-Entropy
- **Batch Size:** 64 Model summary (layer-by-layer output from model.summary())

4.3 Confusion Matrix

- Code snippets for data preprocessing or augmentation
- Confusion matrix visualisation for class-wise accuracy

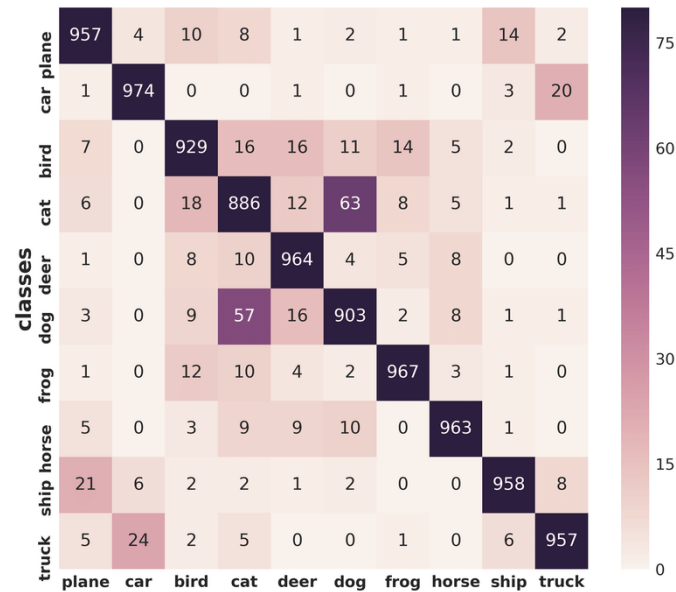


Figure 4: Confusion matrix

5. Training Strategy

5.1 Epochs and Learning Policy

The model trained for a maximum of 300 epochs, but with early stopping enabled to prevent over-training. Learning rate scheduling and dropout layers ensured controlled convergence.

5.2 Callbacks and Regularisation

The following callbacks were employed:

- ReduceLROnPlateau: decreased the learning rate when validation loss plateaued.
- EarlyStopping: stopped training when performance stopped improving.
- ModelCheckpoint: saved the best model weights automatically.


```

Epoch 1/300
704/704 - 79s - 112ms/step - accuracy: 0.3328 - loss: 2.3362 - val_accuracy: 0.4282 - val_loss: 1.8102 - learning_rate: 5.0000e-04
Epoch 2/300
704/704 - 47s - 67ms/step - accuracy: 0.4687 - loss: 1.7236 - val_accuracy: 0.5374 - val_loss: 1.4021 - learning_rate: 5.0000e-04
Epoch 3/300
704/704 - 44s - 63ms/step - accuracy: 0.5302 - loss: 1.5212 - val_accuracy: 0.5768 - val_loss: 1.3299 - learning_rate: 5.0000e-04
Epoch 4/300
704/704 - 46s - 65ms/step - accuracy: 0.5740 - loss: 1.3852 - val_accuracy: 0.6040 - val_loss: 1.2752 - learning_rate: 5.0000e-04
Epoch 5/300
704/704 - 45s - 64ms/step - accuracy: 0.6104 - loss: 1.2540 - val_accuracy: 0.6500 - val_loss: 1.1484 - learning_rate: 5.0000e-04
Epoch 6/300
704/704 - 45s - 64ms/step - accuracy: 0.6435 - loss: 1.1497 - val_accuracy: 0.6988 - val_loss: 0.9542 - learning_rate: 5.0000e-04
Epoch 7/300
704/704 - 46s - 65ms/step - accuracy: 0.6698 - loss: 1.0747 - val_accuracy: 0.6904 - val_loss: 0.9961 - learning_rate: 5.0000e-04
Epoch 8/300
704/704 - 45s - 65ms/step - accuracy: 0.6927 - loss: 1.0124 - val_accuracy: 0.7212 - val_loss: 0.9143 - learning_rate: 5.0000e-04
Epoch 9/300
704/704 - 45s - 64ms/step - accuracy: 0.6998 - loss: 0.9960 - val_accuracy: 0.7480 - val_loss: 0.8695 - learning_rate: 5.0000e-04
Epoch 10/300
704/704 - 46s - 65ms/step - accuracy: 0.7122 - loss: 0.9648 - val_accuracy: 0.7454 - val_loss: 0.8665 - learning_rate: 5.0000e-04
Epoch 11/300
704/704 - 47s - 67ms/step - accuracy: 0.7268 - loss: 0.9343 - val_accuracy: 0.7750 - val_loss: 0.7848 - learning_rate: 5.0000e-04
Epoch 12/300
704/704 - 46s - 66ms/step - accuracy: 0.7365 - loss: 0.9087 - val_accuracy: 0.7616 - val_loss: 0.8491 - learning_rate: 5.0000e-04
Epoch 13/300
...
704/704 - 45s - 64ms/step - accuracy: 0.9383 - loss: 0.2914 - val_accuracy: 0.9152 - val_loss: 0.3897 - learning_rate: 1.0000e-05
Epoch 300/300
704/704 - 44s - 62ms/step - accuracy: 0.9363 - loss: 0.2932 - val_accuracy: 0.9132 - val_loss: 0.3897 - learning_rate: 1.0000e-05
Restoring model weights from the end of the best epoch: 298.

```

Figure 5: Learning rate changes over epochs to show schedule impact

5.3 Hardware and Environment

Training was conducted in TensorFlow/Keras with GPU acceleration. Libraries used include NumPy, Matplotlib, and scikit-learn for metrics and plotting.

6. Performance Evaluation and Results

6.1 Comparative Accuracy:

The model was evaluated on accuracy, loss, and qualitative prediction checks. Final test results:

- **Test Accuracy:** 91.03%
- **Test Loss:** 0.4118

```

313/313 ————— 3s 10ms/step - accuracy: 0.9113 - loss: 0.4154

Test Accuracy: 0.9103000164031982
Test Loss:      0.4117858409881592

```

Figure 6: Comparative Accuracy

6.3 Inference Example

To verify model usability, a random test image (truck) was passed through the trained model. The prediction matched the true label, showing reliable inference capability.



Figure 7: Prediction

7. Comparative Discussion

7.1 Comparison with Classical ML (Track 1)

Traditional methods such as SVM and KNN require manual feature extraction and cannot capture spatial hierarchies. In contrast, CNNs automatically learn patterns directly from pixel data, resulting in higher accuracy and scalability.

7.2 Comparison with Advanced ML (Track 3)

While self-supervised and NAS approaches offer automation, they demand more computational resources and data. The CNN used here provided an interpretable and efficient balance between performance and complexity.

7.3 Strengths and Limitations

Strengths:

- High accuracy and stable convergence
- Effective use of augmentation and dropout
- Robust performance on unseen data

Limitations:

- No per-class F1 analysis
- Limited interpretability compared to attention-based models
- Could benefit from transfer learning or deeper architectures

8. Conclusions and Lessons Learned

8.1 Key Findings

This project successfully implemented a CNN that achieved over 91% accuracy on CIFAR-10, demonstrating the effectiveness of deep learning in image recognition. Data augmentation, batch normalisation, and regularisation proved essential for performance stability.

8.2 Lessons Learned

- Data diversity improves model generalisation.
- Learning rate scheduling significantly affects convergence.
- Overfitting can be managed effectively with dropout and early stopping.

8.3 Future Work

To enhance results, future experiments may include:

- Implementing ResNet or WideResNet architectures.
- Using MixUp or CutMix augmentations.
- Exploring transfer learning from large-scale datasets like ImageNet.

9. References

- Krizhevsky, A., Nair, V. & Hinton, G. (2009). *CIFAR-10 Dataset*. University of Toronto.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press.
- He, K., Zhang, X., Ren, S. & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. CVPR.
- Ioffe, S. & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. arXiv:1502.03167.
- Kingma, D.P. & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. arXiv:1412.6980.